预习 4.3~4.4

4.3 L属性定义的自上而下计算

- 4.3.1 L属性定义
  - 定义
    - 语法制导定义是L属性的，若每个产生式 `A → X₁X₂…Xₙ` 的语义规则：
      - 1. 要么计算A的综合属性
      - 2. 要么计算 `Xj（1 ≤ j ≤ n）` 的继承属性，且仅依赖：
        - 该产生式中Xj左边符号 `X₁,X₂,…,Xj-1` 的属性
        - A的继承属性
  - 性质：S属性定义属于L属性定义
  - 关键问题：L属性的继承属性无法在归约时计算，需关注语义规则的执行时机
  - 示例：变量类型声明的语法制导定义（类型信息从左向右流）
- 4.3.2 翻译方案
  - 定义：语义动作放在{}内，可插入产生式右部任意位置，与分析过程交错执行
  - 执行时机：动作在α推导/归约结束后、β推导/归约开始前执行（可将动作视为文法符号）
  - 设计限制（受L属性启发）
    - 1. 产生式右部符号的继承属性，必须在作用于该符号的动作前计算
    - 2. 动作不能引用其右边符号的综合属性
    - 3. 左部非终结符的综合属性，需在所有引用属性计算完成后计算（通常放在产生式右部末尾）
  - 示例：中缀表达式转后缀表达式、排版单元大小/高度计算
  - 补充：消除左递归可能引入继承属性
- 4.3.3 预测翻译器的设计
  - 基础：基于适合预测分析的翻译方案（LL(1)文法）
  - 核心算法：构造预测翻译器
    - 1. 为每个非终结符A构造函数
      - 形式参数：A的继承属性
      - 返回值：A的综合属性
      - 局部变量：保存产生式中各文法符号的属性
    - 2. 函数代码框架：与预测分析过程一致，根据当前输入选择产生式
    - 3. 代码生成规则（从左到右处理产生式右部）
      - 终结符/属性记号：保存属性值、匹配输入、推进指针
      - 非终结符：调用对应函数，传入继承属性变量，接收综合属性返回值
      - 语义动作：复制代码，将属性引用改为局部变量
  - 示例：表达式文法消除左递归后，构造递归下降翻译器
- 4.3.4 用综合属性代替继承属性
  - 背景：S属性定义比L属性更易理解和实现
  - 核心方法：重写文法，使属性信息流与归约方向一致
  - 示例：Pascal变量声明
    - 原文法：类型信息从右向左流，需继承属性
    - 修改后文法：将类型作为标识符表的最后成分，改为从左向右流，可用综合属性实现
  - 核心思想：调整文法，让属性计算可在归约时完成

4.4 L 属性的自下而上计算

- 4.4.1 删除翻译方案中嵌入的动作
  - L 属性定义常需要**在产生式中间嵌入语义动作**，而自下而上分析只能在**归约时执行动作。**
  - 解决办法：
    - 引入**标记非终结符 M、N 等**，它们只推空串 ε；
    - 把原嵌入动作移到 `M→ε` 的右边；
    - 文法不变、语义动作等价，但**所有动作都移到产生式右端**，可以在归约时执行。
  - 本质：**把分散动作变成只在归约点执行**，适配 LR 分析。
- 4.4.2 分析栈上的继承属性
  - 自下而上分析栈中天然保存**综合属性**；继承属性需要从**左边或上方结点**获取。
  - 核心思路：
    - 若继承属性是**复写规则** `（Y.i = X.s）`，只要**X.s 在栈中的位置静态可确定**，就可以直接取栈中值代替继承属性。
  - 示例：变量声明 int p, q, r
    - `D → T L`
    - L 继承 T.type，且 T.type 在栈中位置固定
    - 归约时直接访问 `stack[top-1].val` 或 `stack[top-3].val` 即可拿到继承值。
  - 结论：**复写规则 + 栈位置可预测 → 可在自下而上分析中直接用继承属性。**
- 4.4.3 模拟继承属性的计算
  - 继承属性不一定只是简单复写，也可能是函数：`C.i = f (A.s)`。
  - 处理方法：
    - 插入**标记非终结符** `N→ε`；
    - 把函数计算 `f (A.s)` 放到 N 的归约动作里；
    - N.s 保存结果，后面结点再从 N.s 继承；
    - 保证**继承值在栈中位置固定。**
  - 排版语言例子：
    - 插入 L、M、N 三个标记非终结符；
    - L 初始化顶层属性；
    - M 传递复写型继承属性；
    - N 完成 `shrink ()` 等函数型继承属性计算；
    - 所有继承属性都变成**复写规则 + 栈固定位置。**
- 适用范围与局限性
  - **LL (1) 文法上的 L 属性定义**：
    - 总能通过加标记非终结符，转成可在 LR 中实现的翻译方案，且文法保持 LR (1)。
  - **LR (1) 文法上的 L 属性定义**：
    - 加入标记非终结符**可能引入移进 - 归约冲突**，不再是 LR (1)，存在实现限制。
- 难点
  - 理解**继承属性**与**栈位置**的对应关系，为什么某些继承值位置固定。
  - 标记非终结符的**插入位置、数量、属性传递逻辑**容易混淆。
  - 函数型继承属性如何**拆成标记非终结符 + 归约动作。**
  - 为什么 LL (1) 上总能成功、LR (1) 上会失败（冲突来源）。

by 幕布用户0301723 由 幕布 发布